Hyperiondev

TASK

HTTP Requests and Responses

Visit our website

Introduction

This task introduces you to HTTP requests and responses, a core concept in web development that powers communication between browsers and servers. By understanding how requests (like loading a webpage) and responses (like receiving that page's content) work, you'll gain the essential context needed to debug, optimise, and build web applications effectively. Whether you're crafting APIs or troubleshooting network issues, this knowledge forms the foundation for creating dynamic, interactive web experiences.

OVERVIEW OF HTTP

HTTP (Hypertext Transfer Protocol) is the foundational protocol of the World Wide Web, establishing a standardised set of rules for communication between clients (like web browsers) and servers. It defines how messages are formatted, transmitted, and processed, ensuring seamless interaction across the web.

For example, when you enter a URL (**Uniform Resource Locator**) into a browser, the browser sends an HTTP request to the web server, asking for the associated webpage. The server responds, often by sending back an HTML file, which the browser interprets and displays to the user. This simple interaction highlights HTTP's role in enabling the smooth exchange of information online.

CLIENT–SERVER COMMUNICATION AND HTTP MESSAGES

HTTP also refers to the communication between clients and servers. Imagine you're sitting at your computer using a web browser (the client) to search for a website. The server is like a library, a powerful computer or system that stores the website's content, like HTML files, images, or data. When you click on a link, your browser sends an HTTP request message asking the server for the content. The server then responds with an HTTP response message, delivering the requested resource so your browser can display it. These messages ensure smooth and clear communication between your computer and the server.

HTTP messages consist of multiple lines encoded in ASCII and rely on a secure Transmission Control Protocol (TCP) connection. TCP, a transport layer protocol in computer networking, ensures reliable and ordered delivery of data between devices. In newer versions of HTTP, messages are sent in smaller, more efficient units called frames, designed to optimise performance and improve speed.

The illustration below shows how clients interact with a server and how that server interacts with the database so that data is processed by the server and sent back to the clients.

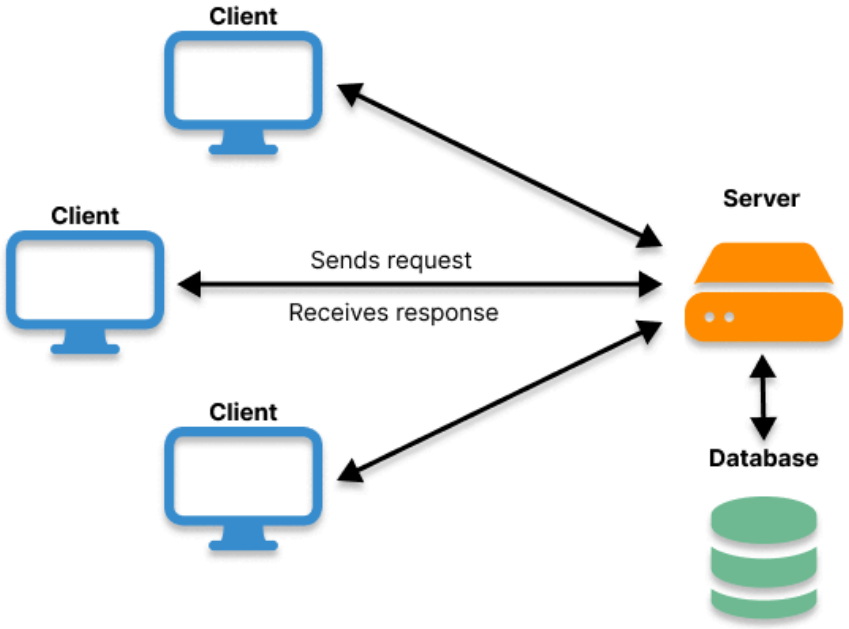


Diagram of client-server communication

It is not entirely accurate to say that a server is a computer that clients make requests to. A computer needs to have appropriate server software running on it in order to make it a server. Examples of server software are **Apache**, **Tomcat**, and **NGINX**.

A client isn't just a device making requests to a server; it also requires specific software to initiate those requests. The most common client you likely use every day is your web browser. However, other applications also act as clients, such as the Netflix app when you're streaming videos. In this case, the Netflix app is the client, and the server delivers the video data to it.

Next, we will dive into the composition of HTTP requests and responses and how they are structured.

BASICS OF AN API

An API (application programming interface) enables software applications to communicate and share data or functionality. In web development, APIs often use HTTP to efficiently facilitate these interactions.

API key concepts:

- Purpose: APIs facilitate data exchange and functionality sharing, such as fetching weather details, updating user profiles, or integrating payment systems.
- Endpoints: These are specific URLs that represent a resource or functionality within the API, such as `/users`, `/orders`, or `/weather`. Each endpoint is designed to handle specific requests.

- Request–response model: APIs use this model where a client (e.g., a browser or application) sends a request to an API server, which processes the request and sends back a response.

Let's dive deeper into how this communication takes place by examining the structure of an HTTP request.

HTTP REQUEST

The client sends an HTTP request message to the server to request a specific action, HTML page, or resource. For example, if a browser, i.e., the client, requests an HTML page, then the server returns an HTML file. An HTTP request is composed of methods, URLs, headers, and a body. Let’s look a little deeper into each of these elements.

1. Methods

The HTTP request method is specified in the request line. Common HTTP methods include PUT, POST, GET, DELETE, HEAD, PATCH, and others, each targeting a specific URL or resource path. Each method serves a unique purpose, and their functions differ. Below is an explanation of some of these methods:

- **GET** retrieves resources from a server, such as web pages, images, documents, etc. It does not modify any resources on the server and only reads the data. The data sent in with the GET request is typically limited in size and sent in the URL.
- **POST** can modify the data on the server. It can create, edit, and delete any object or subject on the site. It's commonly used to create new resources, update existing resources, or modify the server's state. It also sends sensitive data not exposed in the URL, such as passwords. Servers also hold the power to control access to POST endpoints to enforce authentication.
- **DELETE** permanently removes a specific resource on the server. When a delete request is made, the client instructs the server to delete a resource identified by a URL. It also contains no response body.

In simple terms, an endpoint is a specific address (URL) in an API where an application or client can request data or perform actions. Think of it like a door that you open to access a particular service or resource. The API is a set of rules that allows different software applications to communicate with each other. An API provides several endpoints, each for different actions, such as getting data or sending data. For example, an endpoint might allow you to retrieve user information or update a profile, depending on what the application needs.

Each endpoint is associated with a specific HTTP method (such as GET, POST, PUT, DELETE, etc.), which determines the type of operation that the client can perform on the resource. When a client makes an HTTP request to an API endpoint, the API server processes the request and responds with the appropriate data or performs the requested action.

Look here for a more exhaustive **list of HTTP request methods**.

2. URL

A URL is the specific address used to locate resources on the web. It tells a browser or other client where to find the resource and how to interact with it. A URL can include the following parts:

- **Protocol:** Specifies the communication method, such as **http://** or **https://**.
- **Hostname:** The domain name (e.g., **example.com**) or IP address of the server hosting the resource.
- **Port number (optional):** Specifies the port to connect to on the server (e.g., 80 for HTTP or 443 for HTTPS). If not specified, a default port is used.
- **Path:** Indicates the specific resource on the server (e.g., **/images/photo.jpg**).
- **Query parameters (optional):** Additional details passed to the server as key–value pairs, often used for searches or dynamic content (e.g., **?id=123&lang=en**).
- **Fragment identifier (optional):** Indicates a fragment within the resource, pointing to a specific section or element on the page.

The overall format of the URL becomes:

```
protocol://hostname[:port]/path[?query_parameters][#fragment_identifier]
```

For example: **https://example.com:8080/products?page=2#reviews**

The above URL uses the HTTPS protocol to connect to the domain **example.com** on port 8080, accesses the resource at **/products**, passes query parameters **page=2**, and navigates to the reviews section on the page.

3. Request headers

HTTP request headers provide additional information about the client's request, helping the server process it appropriately, and which are formatted as key-value pairs. Common request headers include:

- **Host:** Specifies the hostname and port number of the server.
- **User-Agent:** Used to identify the client who has made the request.
- **Accept:** Indicates the type of media that the client accepts.
- **Content-Type:** Indicates the type of media that can be included in the response body.
- **Cookie:** Contains cookies to keep information about the session.

- **Referrer:** Specifies the page URL that directs the client to the resource.
- **Accept-Language:** Specifies how the client can accept a response.
- **Authorization:** Gives credentials to access private resources on the server.

4. Request body

The **request body** is an optional part of an HTTP request, used to send data from the client to the server, typically with methods like POST, PUT, or PATCH. It contains the actual content or data the client wants to transmit, such as forms, JSON objects, or files. While it's not needed for every request (e.g., GET requests don't use a body), it is essential for sending data like user input, images, or other resources.

When sending data in the request body, there may be limits on the size of the data, depending on the server's or network's capabilities. Additionally, if the data includes sensitive information, it should be protected using encryption, typically through secure protocols like HTTPS. The format of the data (such as JSON, XML, etc.) is usually decided by the client or specified by the API.

HTTP RESPONSE

An HTTP response is the message the server sends back to the client (e.g., a web browser) after processing a request. It consists of three main parts: a status code that indicates the result of the request, response headers with additional information about the response, and the body which contains the requested data or content.

1. Status code

The status code consists of a brief description of the status. It indicates the outcome of the request, whether the request was a failure or a success, and consist of 3-digit numeric codes. Several types of status codes are fixed for cases of success or failure.

Some types of HTTP status codes are:

Informational (1xx)

- **100 Continue:** Shows that the server has received some part of the request and the client should proceed with sending the rest.

Success (2xx)

- **200 OK:** Shows the request was successful and a response is being returned. See this example of successful status code output:

```
user@air ~ % curl -I https://jsonplaceholder.typicode.com/posts
HTTP/2 200
date: Tue, 03 Dec 2024 08:45:48 GMT
content-type: application/json; charset=utf-8
```

- **201 Created:** Shows the request was a success and the server is creating a new response as a response.
- **204 No Content:** The request was a success but no content on the server can be returned as a response.

Redirection (3xx)

- **301 Moved Permanently:** The resource requested on the server has been moved permanently to another location.
- **302 Found:** The resource requested on the server is found in another location.
- **304 Not Modified:** The resource has not been modified since it was last accessed or requested.

Client error (4xx)

- **400 Bad requests:** There is an error on the client's side, so the server cannot comprehend the request.

- **401 Authorization:** The client needs authorisation to access the resources on the server.
- **404 Not Found:** The resource is not found on the server.

Server errors (5xx)

- **500 Internal Server Error:** An unexpected error is encountered on the server side.
- **503 Service Unavailable:** The server is unavailable to process the request.

Take note:

Explore other types of **HTTP response status codes** and browser compatibility.

2. Response headers

HTTP response headers are key–value pairs sent by the server as a response to an HTTP request. Some of the HTTP response headers include the following:

- **Content-Type:** Provides information regarding the content type in the response body.
- **Content-Length:** Gives information about the size of the content.
- **Set-Cookie:** Sets a cookie on the browser of the client.
- **Last-Modified:** Gives information about the last time the resource was modified.
- **Expires:** Gives the time after which the response will expire.
- **Server:** Indicates the version of the software.
- **Access-Control-Allow-Origin:** Specifies the origins of accessing a resource in case of cross-origin requests.

3. Response body

The body of the response is an optional part of the response. It can take various forms, such as text, XML, JSON objects, etc. The request body is separated from the headers by a blank line.

STRUCTURE OF HTTP REQUESTS AND RESPONSES

HTTP requests and responses have similar structures, each consisting of the following components:

1. Start line:

- `Method: GET`: This indicates that the client is requesting data **from** the server. Other common methods include POST (to send data **to** the server), PUT (to update existing data), DELETE (to delete data), and more.
- `Path: /api/users/123`: This is the specific resource being requested. In this case, it's a user with the ID 123 from an API.
- `HTTP Version: HTTP/1.1`: This specifies the version of the HTTP being used (e.g., HTTP/1.1 or HTTP/2), ensuring compatibility between the client and server.

2. Headers:

- `Host: example.com`: This specifies the domain name of the server processing the request. This is essential for directing the request, especially in shared hosting environments where multiple domains share the same IP address.
- `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36`: This provides information about the client making the request, including the browser type, operating system, and version. Servers can use this information to tailor responses, such as providing mobile-friendly content for mobile browsers.
- `Accept: application/json`: This tells the server that the client prefers the response to be in JSON format. The server will attempt to honour this preference if it supports the requested format.

3. Blank line:

- This separates the headers from the body of the request.

4. Body (optional):

- In the case of a GET request, the request doesn't have a body.. GET requests typically don't send data with the request.
- For methods like POST, PUT, or DELETE, the body can contain data to be sent to the server, such as form data, JSON data, or other content.

What follows is an example of an HTTP request and its corresponding response. Firstly, the HTTP request example output:

```
GET /api/users/123 HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36
Accept: application/json
```

HTTP response example output:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 85

{
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "status": "active"
}
```

Take note:

Please read the Mozilla documentation that breaks down the anatomy of an **HTTP message** with some examples.

Please note this is compulsory reading for this task.

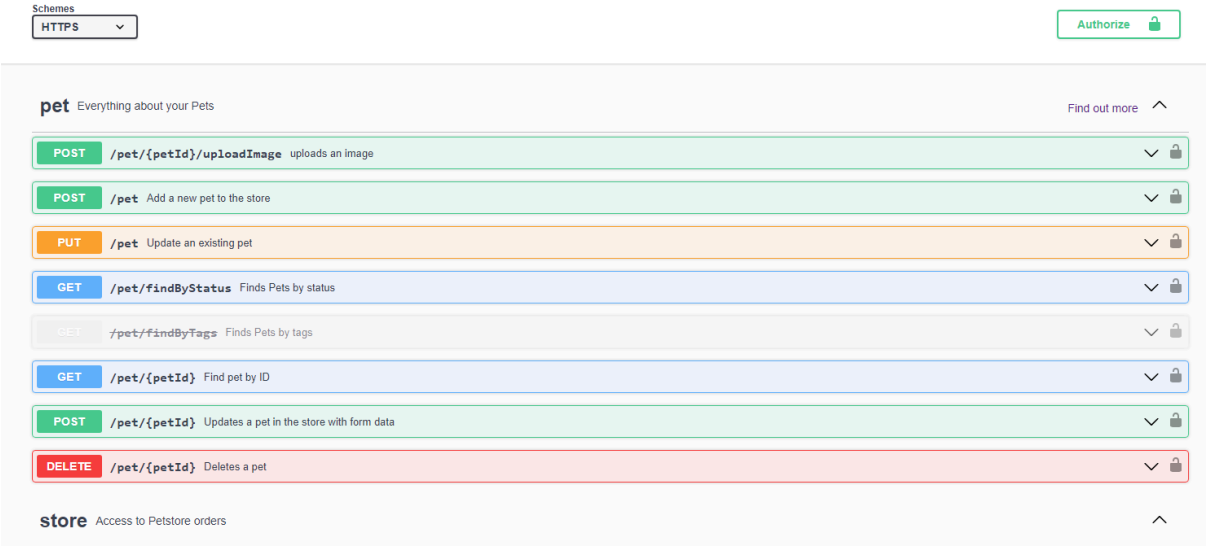
HTTP requests and responses form a communication foundation between clients (front end) and servers (back end). They facilitate the exchange of messages, images, files, etc., giving the user an interactive web experience.

Practical task

In this task, you will explore a sample server and its API endpoints. Then, you will answer questions related to HTTP messages and APIs.

Step 1:

Open your web browser and navigate to the Swagger UI: **https://petstore.swagger.io/**

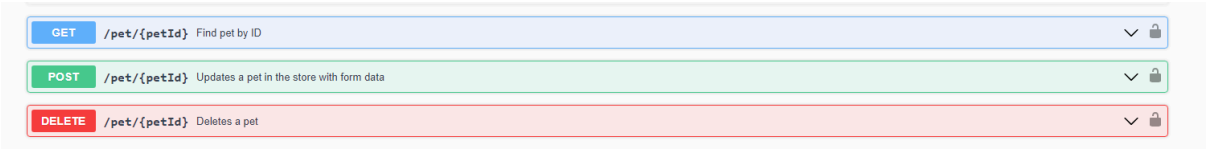


Step 2:

The Swagger UI is already populated with the API definitions of the Petstore API. Explore the API definitions and click on any endpoint (e.g., `GET/pet/{petId}`) to see more details, such as its parameters, responses, and the model of the data it works with.

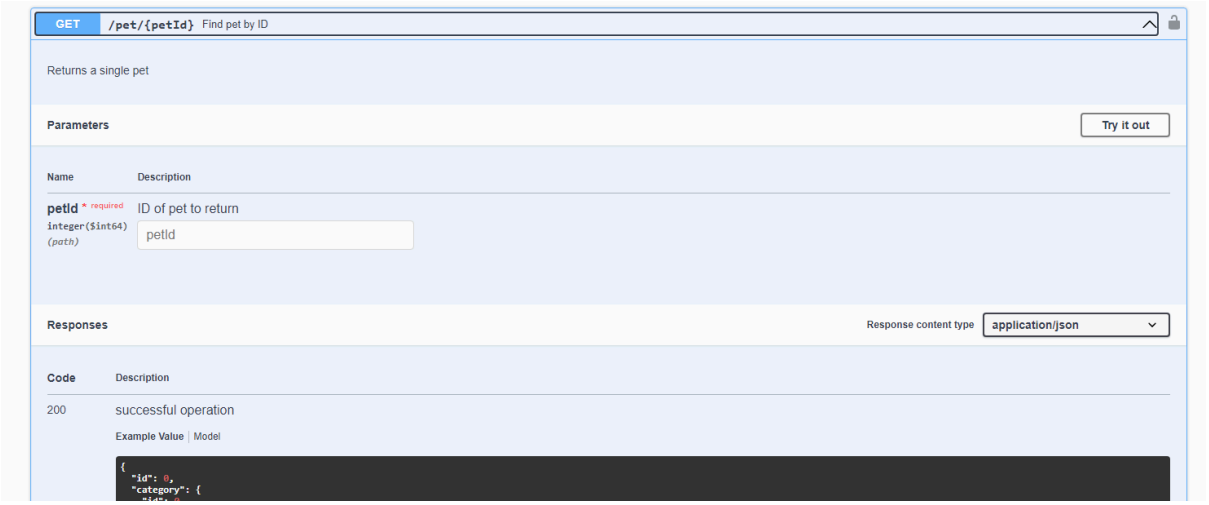
Step 3:

Try executing a request. Click on the `GET /pet/{petId}` endpoint to expand it. Then click the "Try it out" button.



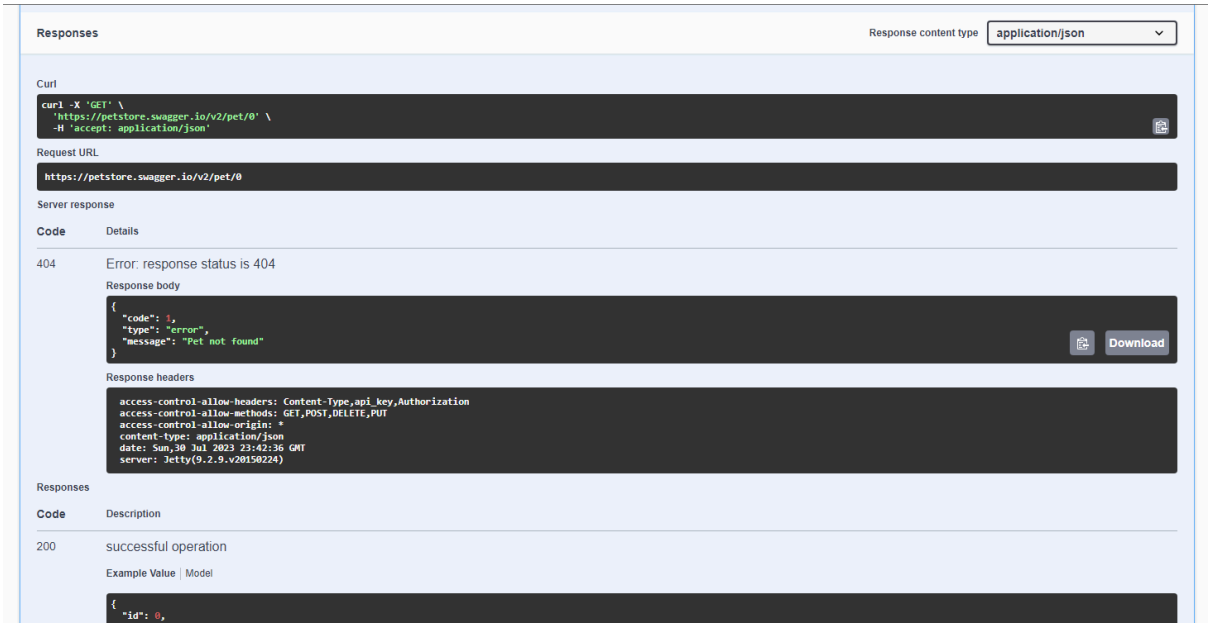
Step 4:

Enter "0" as the `petId` parameter and click "Execute".



Step 5:

Inspect the responses. Swagger UI will show you the curl command that it used to send the request, the URL of the request, the response body, and the response headers.

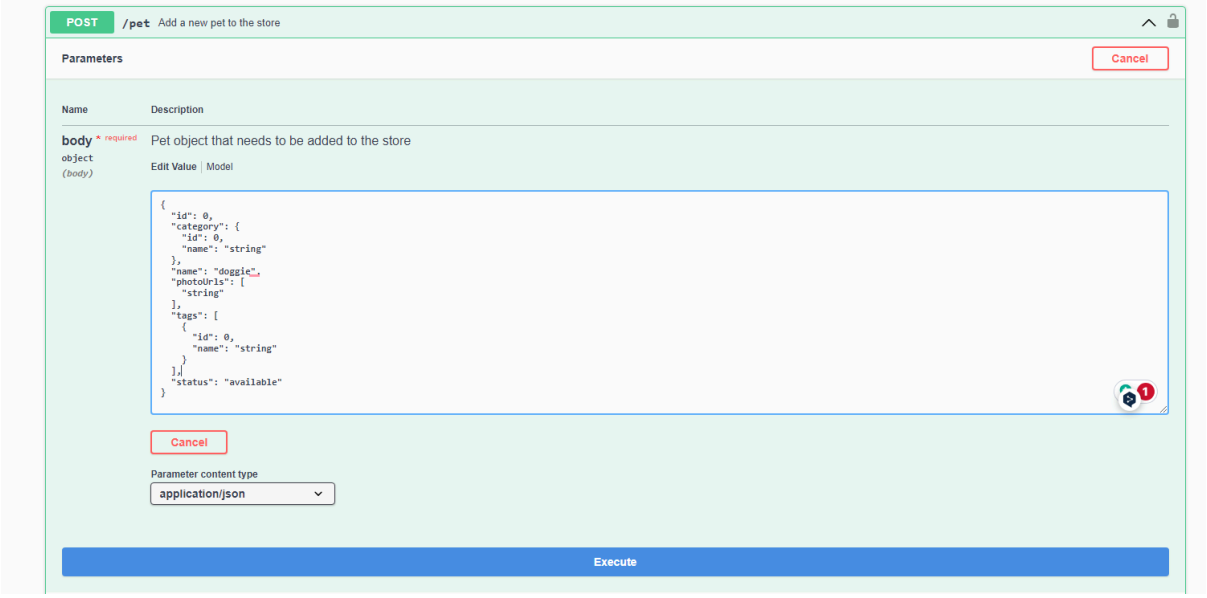


Step 6:

Try executing a different request. Find the "`POST /pet`" endpoint and click "Try it out".

Step 7:

Swagger UI provides an example body for the request. Modify the “name” and “status” fields and then click “Execute”. Again, inspect the responses.



For this task, submit a PDF document titled **SwaggerUI_Petstore**, answering the following questions:

1. What was the response when you sent a `GET` request to `/pet/1`?
2. What was the response code, and what does it mean?
3. What fields were in the response body, and what do they mean?
4. When you sent a `POST` request to `/pet`, what did you put in the request body?
5. What was the response to the `POST` request?
6. What was the response code of the `POST` request, and what does it mean?
7. Can you explain the purpose of the `/pet/{petId}` and `/pet` endpoints?
8. If the `/pet/{petId}` endpoint returned a 404 status code, what would that mean and what might cause it?
9. Choose one more endpoint from the Petstore API and explain its purpose and how you would use it.

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.

Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.